

MUSTAFACODE

Lecture 07

Object Oriented Programming (C++)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 7: Object Oriented Programming (C++)

Chapter 07 — Object Oriented Programming (C++)

07.1 Class

Object Oriented Programming (OOP) ka basic idea **objects** hota hai, aur programming languages mein objects ko implement karne ke liye **classes** use hoti hain.

◆ Class kya hai?

Class C++ ka ek mechanism hai jo real-world objects ko program mein represent karta hai.

👉 Class ek **user-defined data type** hota hai jo:

- attributes (data)
- behaviours (functions)

ko combine karta hai.

Real Life Example

Lion Example

Har lion:

- Color
- Age
- Weight

Behaviours:

- Roar
 - Hunt
 - Run
-

Student Example

Har student:

- Name
- Roll No
- Class

Behaviours:

- Study
 - Register
 - Attend class
-

Real World System Example

University system mein:

- Student
- Teacher
- Subject
- Classroom

Yeh sab objects hain jo ek dusre se interact karte hain.

👉 Example: Student teacher se service leta hai → teaching

07.2 Type in C++

C++ mein built-in types hotay hain:

- int
- float
- char

Lekin **student** jaisa type built-in nahi hota.

👉 Is liye hum **class use karke user-defined type banate hain**

Example:

- Student (student management system)
 - Circle (drawing software)
-

Class Structure

```
class Student {  
    // data members  
    // member functions  
};
```

07.3 Abstraction

◆ Abstraction kya hai?

Abstraction ka matlab: 👉 sirf necessary information show karna
👉 irrelevant details hide karna

Student Example

Included (Important):

- Name
- Address

Not Included (Irrelevant):

- Sibling
- Father Business

👉 System ke liye jo cheez useful nahi hoti wo ignore kar di jati hai.

07.4 User Defined Types

C++ mein 2 ways:

1 Structure

```
struct Student {  
    int rollNo;  
    char name[20];  
};
```

👉 C++ mein struct ke andar functions bhi ho sakte hain.

2 Class

```
class Student {  
private:  
    int rollNo;  
    char *name;  
    float CGPA;  
  
public:  
    void setName(char *);  
    void setRollNo(int);  
};
```

📌 Why Member Functions?

1. Behaviour model karti hain

Object ka behaviour define hota hai.

2. Data Hiding

Data ko direct access se protect karti hain.

3. Validation (Important)

```
void setRollNo(int r) {  
    if (r > 0)  
        rollNo = r;  
}
```

✗ Wrong Example

```
aStudent.rollNo = -5;
```

✓ Correct Way

```
aStudent.setRollNo(5);
```

📌 07.5 Object and Class

♦ Object kya hai?

Object class ka **instance (copy)** hota hai.

Example:

```
Student aStudent;  
Student bStudent;
```

📌 07.6 Accessing Members

1 Dot Operator (.)

```
Student s;  
s.rollNo = 5;
```

2 Arrow Operator (->)

```
Student *s = new Student();  
s->rollNo = 5;
```

Important Note

Direct access OOP principle ke against hai.

👉 Best practice:

- setters
- getters

07.7 Access Specifiers

1 public

✓ outside class access allowed

2 private

✗ only class ke andar access

3 protected

👉 inheritance mein use hota hai

Example

```
class Student {  
    private:  
        char *name;  
        int rollNo;  
  
    public:  
        void setName(char *);  
        void setRollNo(int);  
};
```

Setter Example

```
void Student::setRollNo(int r) {  
    if (r > 0)  
        rollNo = r;  
}
```

Main Function Example

```
int main() {  
    Student aStudent;  
  
    aStudent.setRollNo(10);  
    aStudent.setName("Ali");  
}
```

Wrong Access

```
aStudent.rollNo = 5;    // error (private)
```

Default Access Specifier

Agar kuch mention na ho to:

👉 default = private

Final Summary

- ✓ Class = blueprint of object
 - ✓ Object = instance of class
 - ✓ Abstraction = unnecessary details hide karna
 - ✓ Encapsulation = data + functions + protection
 - ✓ Access Specifiers = control access
 - ✓ Setters/Getters = safe access method
-

Real Life Analogy

Concept	Real Life Example
Class	Blueprint of car
Object	Actual car
Private	Car engine (hidden)
Public	Steering, brakes
Setter	Mechanic adjusting settings